

第8章 C++ I/O 流标准库

在 C++ 编译系统中带有一个面向对象的 I/O 软件包,这就是 I/O 流类库。流是一种抽象,它是对字节结构的数据从一个对象向另一个对象移动的抽象。当程序与外界进行数据交换时,涉及的是程序中的对象和流对象。

本章重点介绍 I/O 流的概念以及 I/O 流类库的结构、使用方法。本章的重点是:了解流的概念;掌握对文本文件的打开、读/写和关闭操作;掌握流插入运算符(<<)及输出流类成员函数 `put()` 的使用;掌握流提取运算符(>>)和输入流类成员函数 `get()`、`getline()` 的使用。

本章只介绍流库的基本概念和简单应用,更详细的内容请参阅使用手册。

8.1 C++ I/O 流概述

在 C++ 语言中没有定义专门的输入/输出(I/O)语句,输入、输出操作是通过 I/O 流实现的。I/O 流库是 C++ 庞大标准库的一个部分。

流是指数据从一个位置流向另一个位置。在 C++ 程序中,数据可以从键盘或外部文件流入到程序的数据结构中(输入),也可从程序中流向屏幕或外部文件(输出)。通常把从流中获取数据的操作(读操作)称为提取操作,而向流中添加数据的操作(写操作)称为插入操作。

尽管 C 语言原来的 `printf()`、`scanf()` 等函数是方便高效的,但不是类型安全的,用户也无法根据需要进行扩展。而 C++ 的 I/O 流是类型安全的,而且 C++ 的 I/O 是可以扩展的,这更符合进行面向对象程序设计的要求。

8.1.1 C++ I/O 流的结构

C++ I/O 流类库是由按面向对象方式建立起来的输入流类和输出流类组成的。I/O 流类库中主要类以及各类之间的关系如图 8.1 和图 8.2 所示。

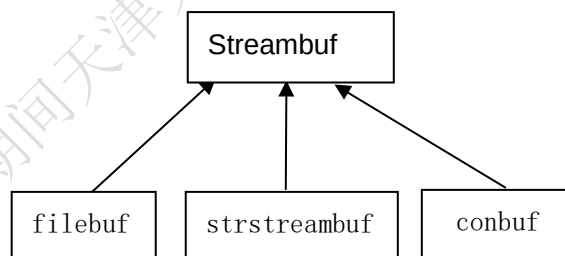


图 8.1 流缓冲区类的结构

图 8.1 表示流缓冲区类的结构。其中, `streambuf` 是一个抽象类。其他类为各种带缓冲的 I/O 流操作提供支持。`filebuf` 是文件缓冲类,用于实现文件的打开、读入、写出和关闭等操作。`strstreambuf` 是字符串缓冲类,支持在内存中进行提取和插入操作。`conbuf` 是控制台缓冲类,支持对屏幕的输出处理。

图 8.2 表示输入、输出流类的结构。其中, `ios` 是一个虚基类,它提供了一些对数据流进行设置的功能。`ostream`、`ofstream` 和 `ostrstream` 是向流中插入数据的操作类;`istream`、`ifstream` 和 `istrstream` 从流中提取数据的操作类;`iostream`、`fstream` 和 `strstream` 是流中数据的综合操作类(插入和提取);`fstreambase` 和 `strstreambase` 分别是文件和字符流类的公共基类。这里列出的不是文件操作的所有的类,但这些类是用于文

件操作的主要的类。

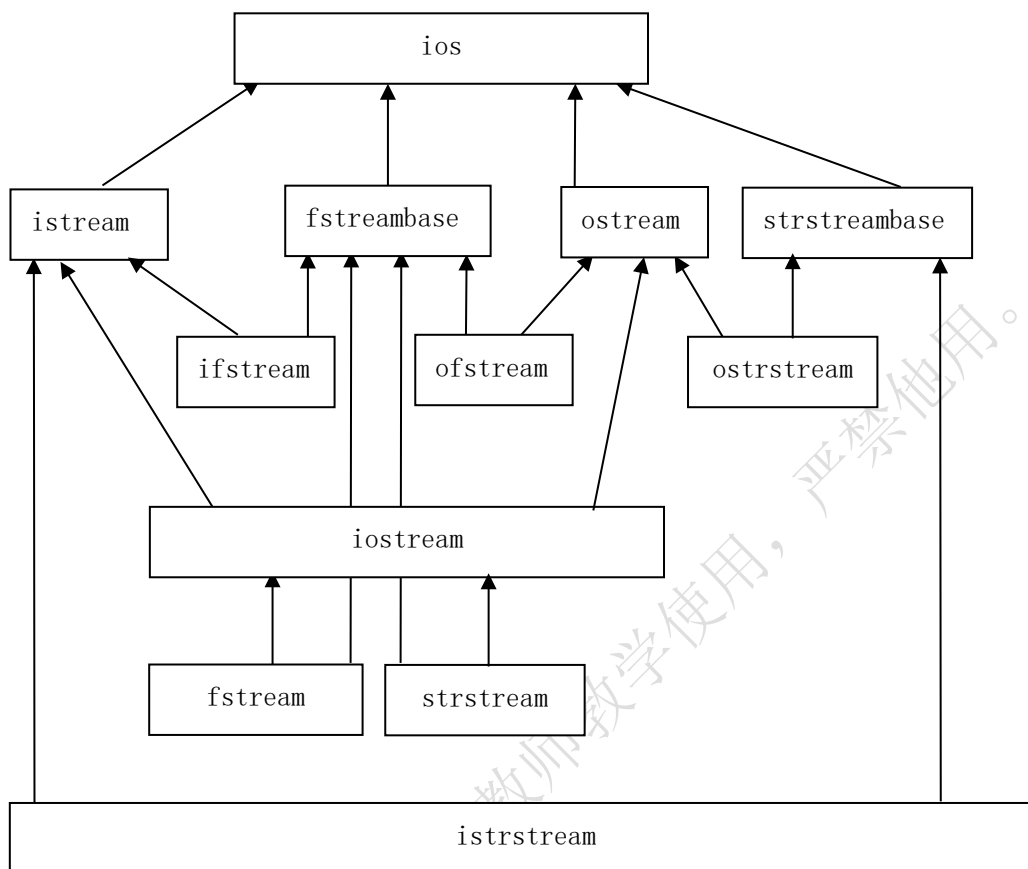


图 8.2 输入输出流类的结构

表 8.1 给出了对这些类的简要说明和类说明所在的类文件。尽管多数 C++ 编译系统还支持很多类,但这里列出的类能够满足对输入、输出的大多数要求。若要了解更多的内容,可参看 C++ 系统中的类库指南。

表 8.1 I/O 流类列表

类 名	说 明	包含文件
ios	流的抽象基类	iostream.h
istream	通用输入流类和其他输入流的基类	iostream.h
ifstream	输入文件流类	fstream.h
ofstream	输出文件流类	fstream.h
iostream	通用输入/输出流类和其他输入/输出流的基类	iostream.h
fstream	输入/输出文件流类	fstream.h

8.1.2 预定义流类的对象与通用的流运算符

用户可以直接使用 C++ I/O 流类中预定义的流类对象和通用的流运算符。

1. 预定义的流类对象

C++ 流类中已预定义了四个对象:

- ①cin 是 istream 类的对象,用于处理标准输入设备(键盘)的输入;
- ②cout 是 ostream 类的对象,用于处理标准输出设备(屏幕)的输出;
- ③cerr 是 ostream 类的对象,用于处理在标准设备上输出出错信息(不带缓冲,即立即显示);
- ④clog 是 ostream 类的对象,用于处理在标准输出设备上输出错误信息(带缓冲),当

缓冲区流插入和提取运算符

重载后的流插入运算符(<<)表示写出数据到流对象中,流提取运算符(>>)表示读取数据到计算机内存中。这两个运算符不仅可以和 cin、cout、cerr 和 clog 这些预定义标准流对象一起使用,而且还可以和用户定义的磁盘文件流对象一起使用。同时,用户还可以重载这些运算符。

8.2 输出流

C++输出流类主要是 ostream 和 ofstream。

ostream 类最适合顺序文本的输出,通常使用 cout、cerr 或 clog 这些预定义的对象实现输出操作。

ofstream 类还支持磁盘文件的输出。C++中,磁盘文件被看做一个字节序列。它可以是一个文本文件,也可以是一个二进制文件,要处理磁盘文件,就需要定义文件输出流对象。

8.2.1 定义文件输出流对象

定义文件输出流对象需要包含 iostream.h 文件和 fstream.h 文件。常用的有两种方法。

1.方法一

先定义文件对象,然后调用 open 成员函数打开该文件。例如:

```
ofstream outfile;           //建立文件输出流对象
outfile.open("data1.txt",ios::out);
```

其中:ios::out 表示文件打开方式为输出(写)方式(这是文件输出流的缺省打开方式)。

用于文件输出流的打开方式还有以下几个:

- ①ios::app,表示追加方式,文件打开时文件指针位于文件尾;
- ②ios::ate,表示打开一现存文件并将文件指针指向文件尾;
- ③ios::nocreate,表示文件存在则打开它,否则该操作失败;
- ④ios::noreplace,表示文件不存在则打开它,文件存在则操作失败;
- ⑤ios::trunc,表示打开文件后清除原有内容,ios::out 隐含此方式;
- ⑥ios::binary,表示以二进制模式打开一个文件(缺省为文本文件)。

可同时指定以上几种方式。例如:

```
ofstream outfile1;
outfile1.open("data1.dat",ios::out|ios::binary|ios::app);
```

这样说明的文件输出流对象 outfile1 所关联的文件 data1.dat 是一个可以在文件尾进行写操作的二进制文件。

2.方法二

在定义文件输出流对象时初始化打开文件。例如:

```
ofstream outfile("data1.txt",ios::out|ios::app);
```

8.2.2 输出流常用成员函数

输出流常用成员函数中除 `open` 函数外,还有 `put`、`write`、`seekp`、`tellp`、`close`、`eof` 等。

(1) `put` 函数

该函数的用法是:

输出流对象名.`put(ch)`

该函数仅输出包含在 `ch` 中的一个字符,且返回当前输出流对象。例如:

```
cout.put('A').put('t');
```

(2) `write` 函数

该函数的用法是:

输出流对象名.`write(s,n)`

该函数输出字符指针 `s` 与所指向的字符串中的 `n` 个字符。当 `s` 所指字符串不足 `n` 时补空格,返回当前输出流对象。例如:

```
cout.write("12345",3).write("ABCDE",7).put('t');
```

输出结果为:

```
123ABCDE t
```

(3) `seekp` 函数

`seekp` 函数的用法是:

输出流对象名.`seekp(流中位置)`

输出流对象名.`seekp(偏移量,参照位置)`

其中,流中位置、偏移量均为 `long` 型,以字节为单位。

参照位置有三种:①`beg(0)`表示相对于流的开始位置;②`cur(1)`表示相对于当前写指

`seekp` 函数是设置文件输出流写指针位置用的,指出下次写出数据的起始位置。利用该指针可实现向磁盘文件随机输出。它们均返回当前输出流对象。

例 8.1 写文本文件示例。

```
#include <iostream.h>
#include <fstream.h>
void main()
{ char *s1="ABCDE",*s2="123456";
  ofstream outfile("data1.txt");           //缺省为 ios::out 方式,文本文件
  outfile.write(s2,3).write(s1,7).put('*'); //用成员函数写入
  char s[30]="C++IO stream";
  outfile.seekp(6,ios::beg);               //移动写指针到 6 字符处,首字符位置为 0
  outfile<<s;                              //用流插入运算符写入
}
```

该程序在 `data1.txt` 文件中开始写入 11 个字符“123ABCDE *”,移动写指针到 6,指向字符 D,接着从这个位置写入 `s` 数组中的字符串,最后文件中的字符为: “123ABCC++ IO stream”。

(4) `tellp` 函数

该函数的用法是:

输出流对象名.`tellp()`

该函数返回当前输出流写指针的位置值。

(5) `close` 函数

在 C++ 中,对磁盘文件操作后均需关闭。该函数的用法是:

输出流对象名.close()

该函数既无参数,也无返回值。当程序正常结束时不用此函数也可以完成同样的作用。

(6) eof 函数

该函数的用法是:

输出流对象.eof()

执行该函数时,若遇到文件结尾的条件,则返回非 0 值。

例 8.2 写入非字符型数据到二进制文件中。

```
#include <iostream.h>
#include <fstream.h>
void main()
{ int a[4]={65,67,97,98},i;
  ofstream ofile("data1.dat",ios::out|ios::binary);
  for(i=0;i<4;i++) ofile.write((char*)&a[i],sizeof(int));
  ofile.close();
}
```

该程序 write()函数首参数处写成(char*)&a[i],目的是把非字符型数据强制按字节写入,以满足 write()函数的要求。

8.3 输入流

C++的输入流类主要是 istream 和 ifstream。前者最适合输入顺序文本,通常使用预先定义的流对象 cin 处理;后者支持磁盘文件的输入。这些文件或是一个文本文件,或是一个二进制文件。与处理磁盘输出文件类似,要处理磁盘输入文件也必须先定义文件输入流对象。

8.3.1 定义文件输入流对象

定义文件输入流对象需要包含 istream.h 文件和 fstream.h 文件。常用以下两种方法。

1. 方法一

先定义文件对象,然后调用 open 函数打开文件,与指定文件建立关联。例如:

```
ifstream infile; //建立文件输入流对象
infile.open("data1.txt",ios::in);
```

其中,ios::in 表示文件打开方式为输入(读)方式。对文件输入流对象来说,这是缺省的打开方式。

用于文件输入流的打开方式还有 ios::binary 和 ios::nocreat。它们的含义与输出流中的说明相同。这些方式可同时使用。例如:

```
ifstream infile1;
infile1.open("data1.dat",ios::in|ios::binary);
```

这样说明的文件输入流对象 infile1 所关联的文件 data1.dat 是一个可以进行读操作的二进制文件。

2. 方法二

在定义文件输入流对象时初始化,打开相关联的文件。例如:

```
ifstream infile("data1.txt",ios::in);
```

实际上,如果文件既要读又要写,还可以把它定义为 fstream 类的对象。例如:

```
fstream iofile("data1.dat",ios::in|ios::out|ios::binary);
```

这样说明后,文件流对象 iofile 即可同时按二进制读和写。也就是说,iofile 既是输入

流对象,又是输出流对象。但应注意 `fstream` 类的对象无缺省打开方式,但其关联文件缺省为文本文件。

8.3.2 输入流常用成员函数

除 `open` 函数外,常用的输入流成员函数还有 `get`、`getline`、`read`、`seekg`、`tellg`、`peek`、`gcount`、`close` 等。

(1) `get` 函数

该函数主要有以下三种使用形式:

输入流对象名.`get()`

这种形式的 `get` 函数是从输入流中获取一个字符(包括空白符)作为函数值。所谓空白符(white space characters)指的是空格符、制表符、换行符以及转义字符 `'\r'`、`'\t'`、`'\f'`、`'\n'`、`'\v'`。

输入流对象名.`get(&c)`

这种形式的 `get` 函数也是从输入流中获取一个字符(包括空白符),并存放在变量 `c` 中。

输入流对象名.`get(s,n,Delim)`

这种形式的 `get` 函数是从输入流中最多获取 `n` 个字符(包括空白符),并存放在字符数组 `s` 中,缺省结束符 `Delim` 为换行符,且自动将换行符转换为字符串结束标志符插入到字符串尾。

该函数返回当前输入流对象。

此函数的结束条件除了上面提到的已读入 `n` 个字符、遇到换行符或指定结束符外,当遇到流输入结束符 `ctrl+z(EOF)` 时也结束(UNIX 和 Linux 中为 `Ctrl+d`)。

例 8.3 `get` 函数使用示例。

```
#include <iostream.h>
void main()
{ char ch, s[80];
  while ((ch=cin.get())!='\n') cout<<ch;          //从键盘输入一行字符并输出
  cout<<endl;
  do{ cin.get(ch); cout<<ch; } while(ch!='\n'); //从键盘输入一行字符并输出
  cin.get(s,80);                                //从键盘输入一行字符并输出
  cout<<s<<endl;
}
```

执行示例为:

I am a student.	//键盘输入(Enter 键结束)
I am a student.	//屏幕输出
You are a worker.	//键盘输入(Enter 键结束)
You are a worker.	//屏幕输出
We are learning the C++ language.	//键盘输入(Enter 键结束)
We are learning the C++ language.	//屏幕输出

(2) `getline` 函数

该函数的用法是:

输入流对象名.`getline(buf,n,Delim)`

该函数最多获取 `n` 个字符并存入字符指针 `buf` 所指向的存储区中,缺省结束符 `Delim` 为换行符,且自动将换行符转换为字符串结束标志符插入到字符串尾。该函数返回当前输入流对象。

此函数的结束条件除了上面提到的已读入 $n-1$ 个字符、遇到换行符或指定结束符外,当遇到流输入结束符 `ctrl+z(EOF)` 时也结束(`UNIX` 和 `Linux` 中为 `Ctrl+d`)。

`getline()` 函数和类似的 `get()` 函数功能的差别在于: `getline()` 函数从输入流中删除结束符,而 `get()` 函数却把该字符保留在输入流中。所以,读入多行字符时应使用 `getline()` 函数,以免出错。

例 8.4 `cin` 输入与 `cin.getline` 输入的区别。

```
#include <iostream.h>
#include <string.h>
const int SIZE=80;
void main()
{   char s1[SIZE],s2[SIZE];
    cout<<"输入一个英语句子:\n";   cin>>s1;
    cin.getline(s2,SIZE);
    cout<<"用 cin 读入的字符串是:"<<s1<<"字符数:"<<strlen(s1)<<endl;
    cout<<"用 cin.getline 读入的字符串是:"<<s2<<"字符数:"<<strlen(s2)<<endl;
}
```

执行示例为:

输入一个英语句子:You are a student.□

用 `cin` 读入的字符串是: You. 字符数:3

用 `cin.getline` 读入的字符串是: are a student. 字符:15

(3)read 函数

该函数的用法是:

输入流对象名. `read(buf,n)`

该函数从输入流中读取 n 个字节(含换行符在内)或遇到输入流结束符(`ctrl+z`)时结束操作,读入后存入字符型指针 `buf` 所指存储区,且不在字符串尾添加空字符(`'\0'`)。该函数返回当前输入流对象。

例 8.5 同时读写一个二进制文件。

```
#include <iostream.h>
#include <fstream.h>
void main()
{   int a[5]={4,2,0,6,5},i;
    fstream iofile("data3.dat",ios::in|ios::out|ios::binary); //定义一个 fstream 类对象
    for(i=0;i<5;i++)           //把数组 a 中的 5 个整数按字节写入文件
    {   iofile.write((char*)&a[i],sizeof(int));
        a[i]=-1;
    }
    iofile.seekg(0,ios::beg);    //设置读指针
    for(i=0;i<5;i++)           //把文件中的 5 个整数按字节读入到数组 a
    {   iofile.read((char*)&a[i],sizeof(int));cout<<a[i]<<" ";
        cout<<endl;
    }
}
```

输出结果为:

4 2 0 6 5

(4)seekg 函数和 tellg 函数

它们是设置文件流读指针和返回当前文件流读指针位置的函数。用法与 `seekp` 和 `tellp` 函数相同，即

```
输入流对象名.seekg(流中位置)
输入流对象名.seekg(偏移量,参照位置)
输入流对象名.tellg()
```

例 8.6 随机访问文件示例。

```
#include <iostream.h>
#include <fstream.h>
#include <stdlib.h>
void main()
{ fstream file1;char fn[]="text1.txt";
  file1.open(fn,ios::in|ios::out);      //同时进行读写操作的文本文件
  if(!file1)                            //打不开文件时 file1 中为 0
  { cout<<"不能打开文件"<<fn<<endl;
    exit(1);                            //终止执行程序
  }
  char txt[]="abcdefgh1234567890 \n";
  for(int i=3;i<sizeof(txt)-4;i++)
  file1.put(txt[i]);                    //每次存入一个字符,将 defgh12345678 存入文件中
  file1.seekg(3);                      //设置读指针为 3,即指向字符 g
  char ch;
  while((ch=file1.get())!='\4') cout<<ch;    //每次读入一个字符并显示
  cout<<endl;
  file1.close();
}
```

输出结果为:

gh123

(5) `peek` 和 `gcount` 函数

函数的用法是:

```
输入流对象名.peek()
输入流对象名.gcount()
```

它们返回的都是一个整数值。前者返回的是输入流中下一个即将被读取的字符,后者返回的是最近一次用 `getline` 或 `read` 函数实际从输入流中读取的字符个数(对 `getline` 函数来说,应包含被自动替换成空字符的换行符在内)。

例 8.7 统计从键盘上输入的 5 行中每行的字符个数,并选出其中最长的字符串。

```
#include <iostream.h>
#include <string.h>
const int SIZE=80;
void main()
{ char s[SIZE],sm[SIZE];
  int cntchars,lenmax=-1,i;
  cout<<"输入 5 行字符:\n";
  for(i=0;i<5;i++)
  { cin.getline(s,SIZE);                //输入一行
```



```

        cntchars=cin.gcount();           //统计当前行字符数
        if(cntchars>lenmax)
        { lenmax=cntchars;               //求最长的行
          strcpy(sm,s);                   //将最长的行存入 sm
        }
        cout<<"行"<<i<<":字符数="<<cntchars<<endl;
    }
    cout<<"最长的行字符数="<<lenmax<<endl;
    cout<<"最长的行为:"<<sm<<endl;
}

```

执行示例为:

输入 5 行字符:

8 荣 8 耻铸民魂

行 0:字符数=13

坚持科学发展观

行 1:字符数=15

0123456789ABCDEF

行 2:字符数=17

好好学习天天向上

行 3:字符数=17

C++面向对象程序设计

行 4:字符数=20

最长的行字符数=20

最长的行为:C++面向对象程序设计

例 8.8 统计从键盘上输入的一行字符中“er”连续出现的次数。

```

#include <iostream.h>
void main()
{ char ch;int cnt=0;
  cout<<"输入一串字符(含多个 er):\n";
  while((ch=cin.get())!='\n') if(ch=='e'&& cin.peek()=='r') cnt++;
  cout<<cnt<<endl;
}

```

执行示例为:

输入一串字符(含多个 er):

erwwpereeereter

5

(6)ignore 和 putback 函数

函数的用法是:

输入流对象名.ignore(n,Delim)

输入流对象名.putback(C)

前者表示跳过输入流中的 n(缺省为 1)个字符或遇到缺省流结束符 Delim(缺省为 EOF)为止(EOF 值为 ctrl+z)。后者表示退回一个字符 C 到输入流中。

它们都返回输入流对象本身。

8.4 格式化输入输出

C++提供两种进行格式化输入、输出的方法:一种是用 `ios` 类的成员函数设置格式;另一种是使用格式操纵算子(manipulators)。

8.4.1 用 `ios` 类成员函数设置格式

1. 格式标志位及其设置

`ios` 类中定义的格式标志字由表 8.2 中的格式标志位组成,它们都属于枚举量。

表 8.2 `ios` 类的格式标志位

标志位名	值	含 义	适用
<code>skipws</code>	<code>0x0001</code>	跳过输入中的空白符	<code>i</code>
<code>left</code>	<code>0x0002</code>	左对齐输出	<code>o</code>
<code>right</code>	<code>0x0004</code>	右对齐输出(缺省设置)	<code>o</code>
<code>internal</code>	<code>0x0008</code>	符号左对齐,值右对齐,二者之间为填充符	<code>o</code>
<code>dec</code>	<code>0x0010</code>	十进制形式数	<code>i/o</code>
<code>oct</code>	<code>0x0020</code>	八进制形式数	<code>i/o</code>
<code>hex</code>	<code>0x0040</code>	十六进制形式数	<code>i/o</code>
<code>showbase</code>	<code>0x0080</code>	数据前显示基类符号 <code>0</code> 或 <code>0x</code> (缺省不显示)	<code>o</code>
<code>showpoint</code>	<code>0x0100</code>	浮点数带小数点输出	<code>o</code>
<code>uppercase</code>	<code>0x0200</code>	大写输出十六进制形式数中的字母和科学记数法中的字母 <code>E</code>	<code>o</code>
<code>showpos</code>	<code>0x0400</code>	正数前显示正号“+”	<code>o</code>
<code>scientific</code>	<code>0x0800</code>	科学记数法形式输出浮点数	<code>o</code>
<code>fixed</code>	<code>0x1000</code>	定点形式输出浮点数	<code>o</code>
<code>unitbuf</code>	<code>0x2000</code>	输入操作后立即刷新流缓冲区	<code>o</code>
<code>Stdio</code>	<code>0x4000</code>	输出操作后清空标准输出和标准错误信息输出	<code>o</code>

操作格式标志位的 `ios` 类的成员函数有以下 4 个:

```
long flags()
long flags(long f)
long setf(long f)
long unsetf(long f)
```

其中, `flags()` 函数返回当前格式标志字; `flags(long f)` 函数用参数值 `f` 更新格式标志字,并返回更新前格式标志字; `setf(long f)` 函数设置由参数值 `f` 所指定的格式标志位,并返回更新前的格式标志字; `unsetf(long f)` 函数清除由参数值 `f` 所指定的格式标志位,并返回更新前的格式标志字。

例 8.9 设置格式标志字示例。

```
#include <iostream.h>
void main()
{   cout.setf(ios::oct);           //设置按八进制输入输出
    cout<<"十进制:94->八进制:"<<94<<endl;
    cout.setf(ios::hex|ios::showbase); //多位同时设置
    cout<<"十进制:94->十六进制:"<<94<<endl;
```

```

        cout.unsetf(ios::showbase);
        cout.flags(ios::uppercase|ios::hex);
        cout<<"十进制:94->十六进制:"<<94<<endl;
    }

```

输出结果为:

```

十进制:94->八进制:136
十进制:94->十六进制:0x5e
十进制:94->十六进制:5E

```

2. ios 类常用格式化输出成员函数

ios 类中定义了多个成员函数,如查询当前流的状态、格式化输出等。这里仅介绍几个常用的格式化输出的成员函数:

```

int width()          (返回当前输出数据时所占字符数(域宽))
int width(int)       (设置输出域宽度)
char fill()          (返回当前所用填充字符(缺省为空格符))
char fill(char)      (设置填充字符,返回设置前的填充字符)
int precision()      (返回当前输出浮点数时的有效数字个数)
int precision(int)   (设置输出浮点数时的有效数字个数,返回设置前的有效数字个数)

```

例 8.10 格式化输出成员函数示例。

```

#include <iostream.h>
void main()
{   cout<<"0123456789012345\n";           //显示字符位置
    cout<<1234<<endl;
    cout.width(12);//设置域宽为 12
    cout<<1234<<endl;
    cout.fill('*');//设置填充符为*
    cout<<1234<<endl;
    cout.flags(ios::left);cout.width(12);
    cout<<1234<<endl;
    cout.flags(ios::right);cout.width(12);
    cout<<1234<<endl;
    cout.precision(5);
    cout<<123.45678<<endl;
}

```

输出结果为:

```

0123456789012345
1234
          1234
1234
1234*****
*****1234
123.46

```

注意,成员函数 width 只对一个输出项格式有效。

8.4.2 使用操纵算子进行格式化

操纵算子实际上是一些特殊的函数,所以可在输入、输出中使用。使用它们要比使用 `ios` 类的成员函数进行格式化更方便。`iostream` 类中的常用操纵算子如表 8.3 所示。

表 8.3 `iostream` 类的常用操纵算子

操纵算子	含义	适用
<code>dec</code>	设置基数为十进制	i/o
<code>hex</code>	设置基数为十六进制	i/o
<code>oct</code>	设置基数为八进制	i/o
<code>endl</code>	插入一换行符,并刷新缓冲区	o
<code>ends</code>	插入一个空字符('\0')	o
<code>ws</code>	提取(跳过)空白符 i flush 刷新流缓冲区	o
<code>setw(int)</code>	设置数据输出域宽	o
<code>setfill(char)</code>	设置填充字符	o
<code>setprecision(int)</code>	设置输出浮点数据的有效数字个数	o
<code>setiosflags(long)</code>	设置参数所指定的格式标志位	i/o
<code>resetiosflags(long)</code>	清除参数所指定的格式标志位	i/o

注 ①各 C++编译系统可能带有不同的操纵算子。

②操纵算子多用于输出。

例 8.11 操纵算子使用示例。

```
#include <iostream.h>
#include <iomanip.h>
void main()
{ cout<<"0123456789012345 \n";
  cout<<setw(12)<<setfill('*')<<1234<<endl
    <<setw(12)<<setiosflags(ios::left)<<1234<<endl
    <<setw(12)<<resetiosflags(ios::left)<<1234<<endl
    <<setw(12)<<setfill('$')<<setprecision(5)<<12.34567<<endl;
}
```

输出结果为:

```
0123456789012345
*****1234
1234*****
*****1234
$$$$$12.346
```

8.5 例题

例 8.12 将文件 `file1.txt` 中的内容拷贝到文件 `file2.txt` 中。

```
#include <iostream.h>
#include <fstream.h>
int main()
{ char ch;
  ifstream fin("file1.txt");
  if(!fin) { cout<<"cannot open file1!";return 1; }
```

```

    cout<<"file1.txt:"<<endl;
    while(fin.get(ch))  cout.put(ch);
    cout<<endl;
    fin.close();
    fin.open("file1.txt");
    ofstream fout("file2.txt");
    if(!fout) { cout<<"cannot open file2!";return 2; }
    while(fin.get(ch))  fout.put(ch);
    fin.close();  fout.close();
    fin.open("file2.txt");
    cout<<"file2.txt:"<<endl;
    while(fin.get(ch)) cout.put(ch);
    cout<<endl;
    fout.close();
    return 0;
}

```

执行示例为:

```

file1.txt:
    ifstream fin("file1.txt");
    if(!fin) { cout<<"cannot open file1!";return 1; }
    ofstream fout("file2.txt");
    if(!fout) { cout<<"cannot open file2!";return 2; }
file2.txt:
    ifstream fin("file1.txt");
    if(!fin) { cout<<"cannot open file1!";return 1; }
    ofstream fout("file2.txt");
    if(!fout) { cout<<"cannot open file2!";return 2; }

```

例 8.13 使用 `get()` 和 `put()` 函数进行文件读写，由命令行参数提供文件名。

```

#include <iostream.h>
#include <fstream.h>
int main(int argc,char*argv[])
{  char ch;
   if(argc<3) { cout<<"bad commcmd!"<<endl; return 1; }
   ifstream fin(argv[1]);
   ofstream fout(argv[2]);
   if(!fin||!fout) { cout<<"cunnot open file!\n"; return 2; }
   while (fin.get(ch))fout.put(ch);
   fin.close();  fout.close();
   cout<<argv[2]<<":\n";
   fin.open(argv[2]);
   while (fin.get(ch)) cout<<ch;
   fin.close();
   return 0;
}

```

注意：本程序在 VC 6.0 中执行前需要选择 project 菜单的“Settings...”命令，在弹出的“project Settings”对话框中选择“Debug”标签，在“Program arguments”文本框中输入命令行参数“file3.txt file4.txt”。设文件 file3 的内容为：

```
12345
YTREW \
Huytr.;
```

执行示例为：

```
file4.txt:
12345
YTREW \
Huytr.;
```

例 8.14 流插入运算符和流提取运算符的重载。

```
#include <iostream.h>
#include <iomanip.h>
class PhoneNo {
    friend ostream & operator<<( ostream&, const PhoneNo & );
    friend istream & operator>>( istream&, PhoneNo & );
private:
    char areaCode[ 4 ];           // 3 位区号
    char exchange[ 5 ];          // 4 位交换机号
    char line[ 5 ];              // 4 位线号
};
//重载流插入运算符
ostream &operator<<( ostream &output, const PhoneNo &num )
{   output << "(" << num.areaCode << ")" << num.exchange << "-" <<
num.line;
    return output;               // 可连续输出: cout<<a<<b
}
//重载流提取运算符
istream &operator>>( istream &input, PhoneNo &num )
{   input.ignore();              // 跳过 (
    input >> setw( 4 )>>num.areaCode;   // 输入区号
    input.ignore( 2 );           // 跳过) 和空格
    input >> setw( 5 )>>num.exchange;   // 输入交换机号
    input.ignore();              // 跳过-
    input >> setw( 5 )>>num.line;       // 输入线号
    return input;               // 可连续输入: cin >> a >> b
}
int main()
{   PhoneNo phone;
    cout << "输入电话号,格式为(123) 2456-7890:\n";   cin >> phone;
    cout << "输入的电话号是: " << phone << endl;
    return 0;
}
```

输出结果为:

输入电话号,格式为(123) 2456-7890:

(022) 8740-1132

输入的电话号是: (022) 8740-1132

仅供疫情期间天津大学教师教学使用，严禁他用。

练习 8

1. 回答下列问题

- (1) 什么是流, 流的提取和插入指什么?
- (2) I/O 流在 C++ 中起什么作用?
- (3) get() 与 getline() 函数有什么异同?
- (4) C++ 中进行格式化输入、输出有哪两种方法?

2. 选择题

(1) cin 是 istream 的一个对象, 处理标准输入; cout 是 ostream 的对象, 处理标准输出。由它们两个派生出一个类_____, 提供双向 I/O 操作。

- A) istream B) iostream C) ifstream D) ofstream

(2) _____ 是处理文件 I/O 操作的类。

- A) istream B) fstream C) ofstream D) iostream

(3) 设有定义: int a, *pa=&a, 用十进制输出 a 的地址值时正确的使用是_____。

- A) cout<<pa; B) cout<<&pa; C) cout<<dec<<pa; D) cout<<long(pa);

(4) 在 I/O 格式控制操作中, _____ 是设置域宽的。

- A) setw() B) setfill() C) setbase() D) ws

(5) 下列函数中, _____ 是对文件进行写操作的。

- A) get() B) put() C) read() D) seekg()

3. 阅读下列程序并写出输出结果

- (1)

```

#include<iostream.h>
#include<iomanip.h>
void main()
{ int i,j,k;
  for(i=1;i<=6;i++)
  { for(j=1;j<=20-3*i;j++) cout<<" ";
    for(k=1;k<=i;k++) cout<<setw(3)<<k;
    for(k=i-1;k>0;k--) cout<<setw(3)<<k;
    cout<<endl;
  }
}

```
- (2)

```

#include <iostream.h>
#include <iomanip.h>
void print(int n)
{ if(n!=0)
  { print(n-1);
    for(int i=1;i<=n;i++) cout<<setw(3)<<i;
    cout<<endl;
  }
}
void main() { int n=4; print(n); }

```
- (3)

```

#include <iostream.h>
#include <iomanip.h>
void main(){ cout<<hex<<65<<" "<<oct<<65<<" "<<dec<<65<<endl; }

```


4. 程序填空题

(1) 以下程序将字符数组 **a** 和 **b** 的元素写到文本文件 **letter.dat** 中。

```
#include <stdlib.h>
#include <stdio.h>
#include <fstream.h>
void main()
{
    _____;
    char a[]="1234", b[]="abcdef";
    outfile.open("letter.dat",ios::out);
    if( _____ ) { cout<<"can not open file:letter.dat!\n"; exit(0); }
    _____<<a<<b;
    _____;
}
```

(2) 以下程序将从 **letter.dat** 中读出的内容显示在屏幕上,并将其中的字母写到文件 **data.dat** 中去。

```
#include <iostream.h>
#include <fstream.h>
int main()
{
    char ch;
    fstream infile,outfile;
    infile.open("letter.dat",ios::in);
    if(!infile) { cout<<"文件 1 不能打开!\n"; return 1; }
    outfile.open( _____ ,ios::out);
    if(!outfile){ cout<<"文件 2 不能打开!\n"; return 2; }
    while(infile.get(ch))
    {
        cout<<ch;
        if(ch>='a'&&ch<='z'||ch>='A'&&ch<='Z')
            _____<<ch;
    }
    cout<<endl;
    infile.close(); outfile.close();
    return 0;
}
```

5. 编写一个程序,将从键盘上输入的一串字符存入一个名为 **file1.txt** 的文件,以“#”作为输入的结束。

6. 使用 I/O 流打开第 5 题建立的文件 **file1.txt**,读出其内容并显示出来。

7. 建立一个通讯录文件。每个人的记录包括编号、姓名、电话号码和地址。程序能根据每个人的编号检索输出该人的姓名、电话和地址。

8. 输入一串有字母和数字混合的字符串,编程序将字符串中的字母和数字分开,分别存入两个不同的文件。

9. 将 1!、2!、3!、……、10! 各值存入 **data.dat** 文件,要求在文件中每个值占一行,以左对齐方式存储。